南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理
COMPUTER ORGANIZATION

本课件内容参考自David A. Patterson与John L. Hennessy的教材《Computer Organization and Design: The Hardware/Software Interface》官方配套课件。

zhangj4@sustech.edu.cn

Lecture 4

RISC-V Instruction Format

zhangj4@sustech.edu.cn

Outline

- Instruction Encoding Formats
- Branch and Jump Addressing
- Large Immediate
- Different ISAs

Instructions as Numbers

- Most data we work with is in words (32-bit chunks):
  - Each register holds a word
  - lw and sw both access memory one word at a time
- So how do we represent instructions?
  - Remember: Computer only represents 1s and 0s, so assembly code "`add x10,x11,x0`" is meaningless to hardware
  - RISC-V seeks simplicity: since data is in words, make instructions be fixed-size 32-bit words also
    - Same 32-bit instruction definitions used for RV32, RV64, RV128

Instructions in Binary

- Divide 32-bit instruction word into “fields”
- Each field tells processor something about instruction
- We could define different set of fields for each instruction, but for hardware simplicity, group possible instructions into six basic types of instruction formats:
  - **R-format** for register-register arithmetic/logical operations
  - **I-format** for register-immediate ALU operations and loads
  - **S-format** for stores
  - **B-format** for branches (SB in textbook)
  - **U-format** for 20-bit upper immediate instructions
  - **J-format** for jumps (UJ in textbook)

RISC-V Instruction Formats

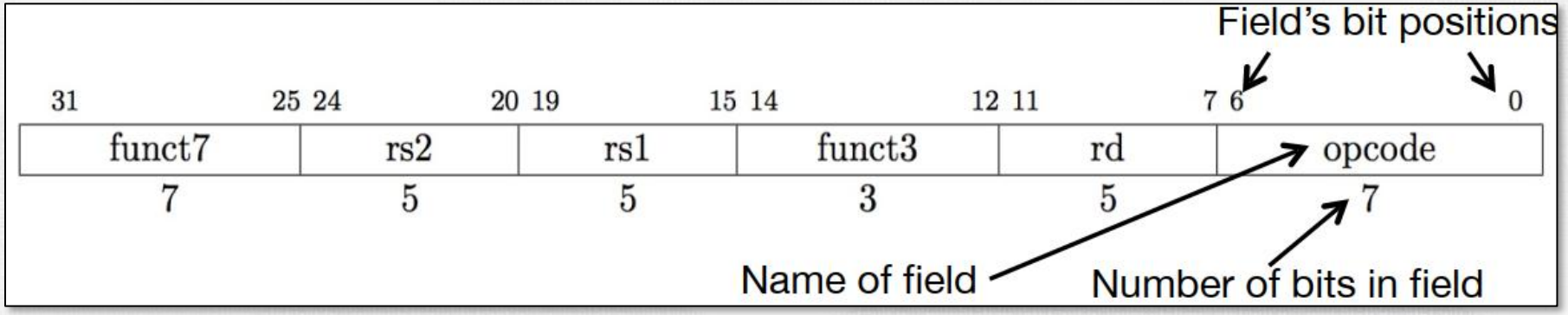
31 30 25	24 21 20	19 15	14 12	11 8	7	6 0	
funct7	rs2	rs1	funct3	rd		opcode	R-type
imm[11:0]		rs1	funct3	rd		opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]		opcode	S-type
imm[12] imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type
imm[31:12]				rd		opcode	U-type
imm[20] imm[10:1]	imm[11]	imm[19:12]		rd		opcode	J-type

R-Type

- R-format is for register-register arithmetic/logical operations

  add x7, x5, x6 add/sub/slt/sltu sll/srl/sra and/or/xor

- Format



- In this format, 32-bit instruction word divided into six fields of differing numbers of bits each field: 7+5+5+3+5+7 = 32 bits
- In this case:
  - opcode is a 7-bit field that lives in bits 0-6 of the instruction
  - rs2 is a 5-bit field that lives in bits 20-24 of the instruction

R-Type

- opcode/funct fields (This information can be found in RISC-V reference card)

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7

- opcode: partially specifies which instruction it is
  - Note: This field for all R-Format register-register arithmetic/logical instructions is 0110011
- funct7+funct3: combined with opcode, these two fields describe what operation to perform
- Question: Why aren't opcode and funct7 and funct3 a single 17-bit field?
  - We'll answer this later

All RV32 R-format instructions

- This information can be found in RISC-V reference card

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

Encoding in funct7 + funct3 selects particular operation

R-Type

- register fields: rs1, rs2, rd

add x7, x5, x6 # x7=x5+x6

rd rs1 rs2

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7

- Each register field (rs1, rs2, rd) holds a 5-bit unsigned integer [0-31] corresponding to a register number (x0-x31)
  - rs1 (Source Register #1): specifies register containing first operand
  - rs2 : specifies second register operand
  - rd (Destination Register): specifies register which will receive result of computation

R-Type Example
RISC-V to Machine Code

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7

- add x18, x19, x10 # rd=x18, rs1=x19, rs2=x10

0000000	01010	10011	000	10010	0110011
ADD	rs2=10	rs1=19	ADD	rd=18	Reg-Reg OP

Answer: 0x00a98933

R-Type Example
Machine Code to RISC-V

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7

- 0x01B342B3
  - Binary form 0000 0001 1011 0011 0100 0010 1011 0011
  - Determine instruction type with lowest 7 bits, 0110011 means R format
  - Then fill bits in each field from right to left

0000000	11011	00110	100	00101	0110011
XOR	rs2=27	rs1=6	XOR	rd=5	Reg-Reg OP

Answer: xor x5, x6, x27

I-Type

- What about instructions with immediates? addi x7, x5, 100
  - Ideally, RISC-V would have only one instruction format (for simplicity): unfortunately, we need to compromise
  - if use R-Format

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7

  - replace rs2 by immediate: 5-bit field only represents numbers up to the value 31: would like immediates to be much larger
- Define another instruction format that is mostly consistent with R-format
  - Note: use rs2 and funct7 together to represent a larger immediate

I-Type

- I-format is for register-immediate ALU operations and loads

  addi/subi/andi/ori/xori/slti/sltui slli/srli/srai lw/lh/lhu/lb/lbu

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7

- Only one field is different from R-format, rs2 and funct7 replaced by 12-bit signed immediate, imm[11:0]
- Remaining field format (rs1, funct3, rd, opcode) same as before
- imm[11:0] can hold values in range [-2^{11} , $+2^{11}-1$]
- Immediate is always sign-extended to 32-bits before use in an arithmetic/logic operation

I-Type Example
RISC-V to Machine Code

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7

- addi x15,x1,-50 # rd=x15, rs1=x1

111111001110	00001	000	01111	0010011
imm=-50	rs1=1	ADD	rd=15	OP-Imm

Answer: 0xfce08793

All RV32 I-Type Arithmetic/Logical Instructions

imm			funct3		opcode	
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI

One of the higher-order immediate bits is used to distinguish "shift right logical" (SRLI) from "shift right arithmetic" (SRAI)

"Shift-by-immediate" instructions only use lower 5 bits of the immediate value for shift amount (can only shift by 0-31 bit positions)

Load Instructions are also I-Type

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7
offset[11:0]	base	width	dest	LOAD

- lw rd, immediate(rs1) # rd = Mem[rs1+imm]

- The 12-bit signed immediate is added to the base address in register rs1 to form the memory address
  - This is very similar to the add-immediate operation but used to create address not to create final result
- The value loaded from memory is stored in register rd

I-Type Load Example: RISC-V to Machine Code

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7

- lw x14, 8(x2)

000000001000	00010	010	01110	0000011
imm=+8	rs1=2	LW	rd=14	LOAD

Answer: (0x00812703)

All RV32 Load Instructions

imm[11:0]	rs1	000	rd	0000011	LB
imm[11:0]	rs1	001	rd	0000011	LH
imm[11:0]	rs1	010	rd	0000011	LW
imm[11:0]	rs1	100	rd	0000011	LBU
imm[11:0]	rs1	101	rd	0000011	LHU

funct3 field encodes size and signedness of load data

- LBU is “load unsigned byte”
- LH is “load halfword”, which loads 16 bits (2 bytes) and sign-extends to fill destination 32-bit register
- LHU is “load unsigned halfword”, which zero-extends 16 bits to fill destination 32-bit register
- There is no LWU in RV32, because there is no sign/zero extension needed when copying 32 bits from a memory location into a 32-bit register

- Exercise: Convert lbu x6, 4(x5) to RISC-V Assembly

S-Type Used for Stores

sw x14, 8(x2) both x2 and x14 are source register

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7	5	5	3	5	7
offset[11:5]	src	base	width	offset[4:0]	STORE

- Convert RISC-V Assembly to Machine Code:
  - sw x14, 8(x2) # rs1=x2, rs2=x14, offset = 8
  - immediate = 8 = 0000_0000_1000

0000000	01110	00010	010	01000	0100011
offset[11:5] =0	rs2=14	rs1=2	SW	offset[4:0] =8	STORE

immediate: 0000000 | 01000 combined 12-bit offset = 8

zhangj4@sustech.edu.cn

All RV32 Store Instructions

imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW

Branch and Jump Addressing

zhangj4@sustech.edu.cn

RISC-V Conditional Branches

- `beq x1, x2, Label # rs1=x1, rs2=x2, label=offset`
  - typically used for if-else, while, for, etc
  - read two registers but don't write any register (similar to stores)
- How to encode the label, i.e., where to branch to?
- We use an immediate to encode PC relative offset
  - If we **don't** take the branch:
    - PC = PC + 4 (i.e., next instruction)
  - If we **do** take the branch:
    - PC = PC + relative offset

PC-Relative Addressing

- PC-Relative Addressing:
  - in byte unit
  - can be positive or negative
  - Use 12-bit immediate to encode offset, which means $\pm 2^{11}$ bytes relative to PC
- RISC-V instructions are 32-bit, Address of instruction is always a multiple of 4 (in bytes)
  - meaning offset's lowest 2 bits are always 0
- To enlarge branching area, RISC-V uses half word unit for offset
  - which means $\pm 2^{12}$ bytes relative to PC
  - We use half word unit but not word, because RISC-V standard also support 16-bit possibility
- **Thus Target address = PC + imm value × 2**

B-Type for Branches

31	30 25	24 20	19 15	14 12	11 8	7	6 0
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
1	6	5	5	3	4	1	7

- B-format is mostly same as S-Format, with two register sources (rs1/rs2) and a 12-bit immediate
- But now immediate represents the branch offset in units of half-words. To convert to units of Bytes, left-shift by 1.
- The 12 immediate bits encode actually 13-bit signed byte offsets (lowest bit of offset is always zero, so no need to store it)
  - Thus the imm[12:1] in the total encoding, compared with imm[11:0] in the I-type encodings

All RV32 B-Type Instructions

imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	B beq
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	B bne
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	B blt
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	B bge
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	B bltu
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	B bgeu

B-Type Example
RISC-V to Machine Code

- RISC-V Assembly:

```
Loop:   beq x19,x10,End
        add x18,x18,x10
        addi x19,x19,-1
        j Loop
End:
```

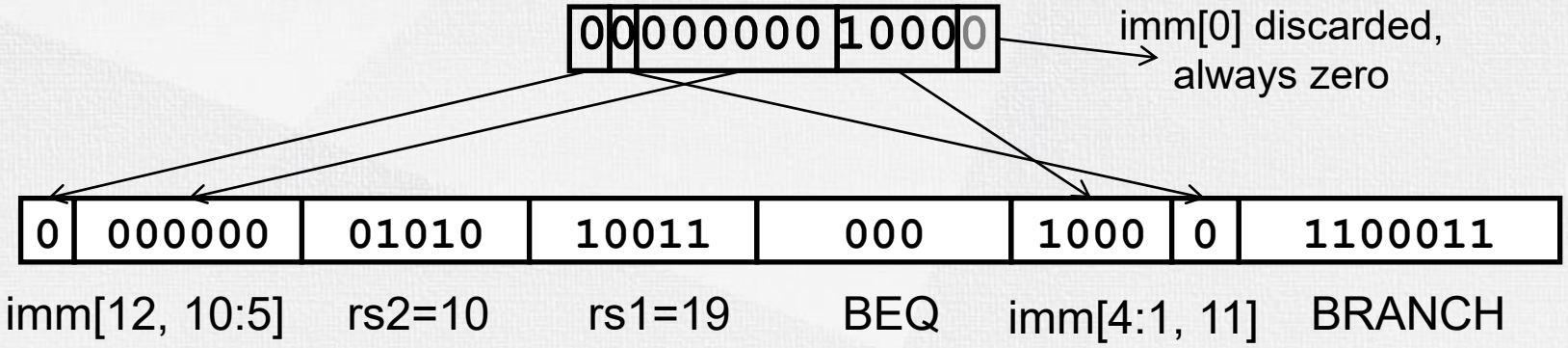
1 2 3 4 Count instructions from branch

- If change label to numeric branch offset = 4 instructions = 16 bytes

```
beq   x19,x10, 16 # offset in bytes = 16
      rs1  rs2  offset
```

  - To encode, we represent 16_{ten} in 13-bit, then take high 12 bits as immediate

13-bit immediate, with value 16_{ten}



B-Type Example
Machine Code to RISC-V

- Convert B format machine to RISC-V Assembly:

0	000000	01010	10011	000	1000	0	1100011
imm[12, 10:5]		rs2=10	rs1=19	BEQ	imm[4:1, 11]		BRANCH

  - imm[12:1] = 0000_0000_1000 (12bits)
  - Target address = PC + imm value[12:1] × 2 = PC + 16
  - Where offset = 16

Answer: beq x19, x10, 16

- Does the value in branch immediate field change if we move the code?
  - If moving individual lines of code, then yes
  - If moving all of code, then no (because PC-relative offsets)

RISC-V Immediate Encoding

Instruction Encodings, inst[31:0]

31 30 25	24 21	20 19 15	14 12	11 8	7	6 0	
funct7	rs2	rs1	funct3	rd		opcode	R-type
imm[11:0]		rs1	funct3	rd		opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]		opcode	S-type
imm[12] imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type

32-bit immediates produced, imm[31:0]

31 30 20	19 12	11	10 5	4 1	0	
— inst[31] —			inst[30:25]	inst[24:21]	inst[20]	I-immediate
— inst[31] —			inst[30:25]	inst[11:8]	inst[7]	S-immediate
— inst[31] —		inst[7]	inst[30:25]	inst[11:8]	0	B-immediate

Upper bits sign-extended from inst[31] always

Only bit 7 of instruction changes role in immediate between S and B

Jump: Branching Far Away

- What do we do if destination is > 2^{10} instructions away from branch?
  - replace conditional jump to unconditional jump

```
beq x10,x0,far
# next instr
…
…
far:
```

→

```
bne x10,x0,next
j far
next: # next instr
…
…
far:
```

  - What exactly is “j far” in basic instruction?
    - jal x0, far (J format, see details in the following slides)

J-Type for Jump Instructions

31	30 … 21	20	19 … 12	11 … 7	6 … 0
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1	10	1	8	5	7
	offset[20:1]			dest	JAL

- `jal rd, Label`
  - `Example: jal, x0, Label # pseudo code is: j Label`
- JAL saves PC+4 in register rd (the return address)
  - "j Label" is pseudo-instruction, uses JAL but sets rd=x0 to discard return address
- Set PC = PC + offset (PC-relative jump)
- Use 20 bits immediate → Target somewhere within $\pm 2^{19}$ half-word offset
- Immediate encoding optimized similarly to branch instruction to reduce hardware cost

Branch & Jump Example

31 30 … 25	24 … 21	20	19 … 15	14 … 12	11 … 8	7	6 … 0	
funct7	rs2		rs1	funct3	rd		opcode	R-type
imm[11:0]			rs1	funct3	rd		opcode	I-type
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type
imm[20]	imm[10:1]	imm[11]	imm[19:12]		rd		opcode	J-type

x11 x9 x10

while (save[i] == k)
i += 1;

i in x9
base add in x11
k in x10
save[i] in x5

Loop: slli x6, x9, 2	#I	80000_{ten}	000000000010_01001_001_00110_0010011
add x6, x6, x11	#R	80004	0000000 01011_00110_000_00110_0110011
lw x5, 0(x6)	#I	80008	000000000000_00110_010_00101_0000011
bne x5, x10, Exit	#B	80012	0_000000_01010_00101_001_0110_0_1100011
addi x9, x9, 1	#I	80016	000000000001_01001_000_01001_0010011
j Loop	#J	80020	1_1111110110_1_11111111_00000_1101111
Exit: …		80024	…

- bne x5,x10,12
  - immediate = 12 (13 bits decimal but encoded high 12 bits in instruction) → 0 0 000000 0110 0
  - bne target = PC+{inst[31],inst[7],inst[30:25],inst[11:8]}x2=80012 + 6*2 = 80024
- jal x0,-20
  - immediate = -20 (21 bits decimal but encoded high 20 bits in instruction)
  - j target = PC+{inst[31],inst[19:12],inst[20],inst[30:21]}x2=80020+(-10*2)=80000

Large Immediate

32-bit Constants

- Most constants are small, 12-bit immediate is sufficient
- For the occasional 32-bit constant, e.g. `li x10 0x87654321,  la x10 my_var`
  - Use LUI, LUI writes the upper 20 bits of the destination with the immediate value, and clears the lower 12 bits.
  - Together with an ADDI to set low 12 bits, can create any 32-bit value in a register using two instructions (lui/addi)
- Example, set x10 to be 0x87654321

```
lui x10, 0x87654   # x10 = 0x87654000
addi x10, x10, 0x321 # x10 = 0x87654321
```

- Corner case: set 0xDEADBEEF (addi 12-bit immediate is always sign-extended, if 12-bit immediate's sign bit is 1, will subtract from upper 20 bits)

```
lui x10, 0xDEADC # x10 = 0xDEADC000
addi x10, x10, 0xEEF # x10 = 0xDEADBEEF
```

U-Type for “Upper Immediate”

31 … 12	11 … 7	6 … 0
imm[31:12]	rd	opcode
20	5	7
U-immediate[31:12]	dest	LUI
U-immediate[31:12]	dest	AUIPC

- Has 20-bit immediate in upper 20 bits of 32-bit instruction word
- One destination register, rd
- Used for two instructions
  - LUI – Load Upper Immediate, rd = imm << 12
  - AUIPC – Add Upper Immediate to PC, rd = PC + (imm << 12)

Jumps to Absolute

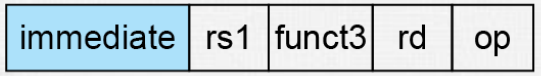
- For even farer jumps, we use jalr
- `jalr rd, rs, immediate / jalr rd, immediate(rs)`
  - Writes PC+4 to rd (return address)
  - Sets PC = rs + immediate (an absolute address)
  - **no** multiplication by 2 (as imm represents absolute address now)
- **Note: jalr is I-Format Instruction**

RISC-V Instruction Formats Summary

31 30 25	24 21 20	19 15	14 12	11 8	7	6 0	
funct7	rs2	rs1	funct3	rd		opcode	R-type
imm[11:0]		rs1	funct3	rd		opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]		opcode	S-type
imm[12] imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type
imm[31:12]				rd		opcode	U-type
imm[20] imm[10:1]	imm[11]	imm[19:12]		rd		opcode	J-type

RISC-V Addressing Summary

1. Immediate addressing

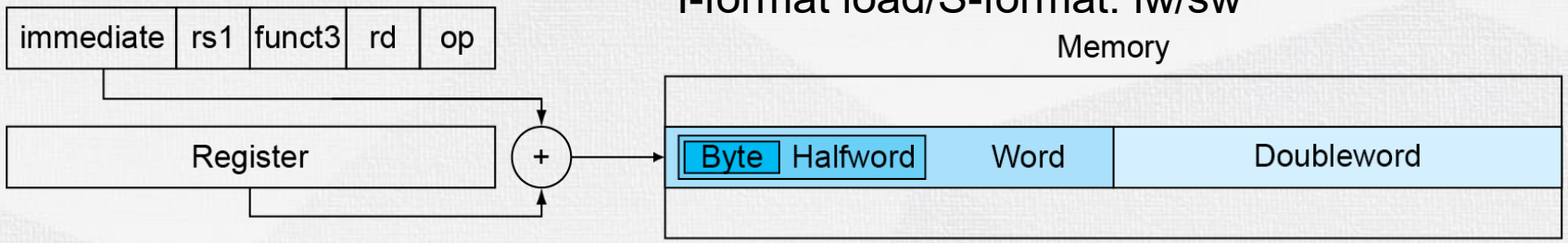


I-format Arithmetic: addi
U-format: lui

2. Register addressing

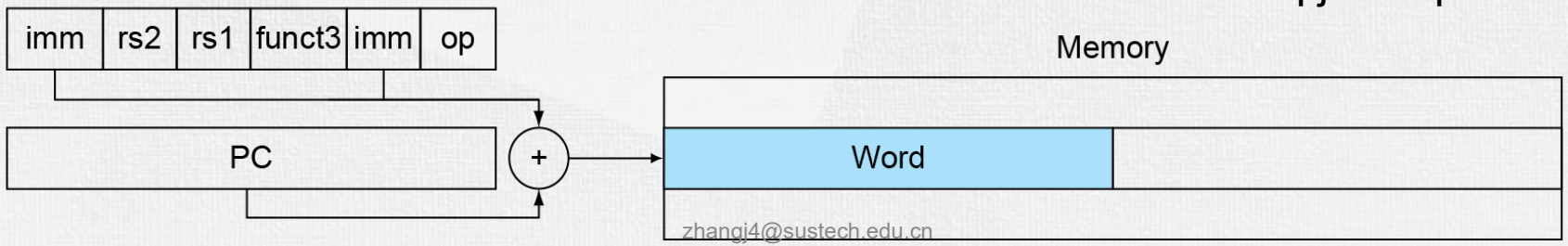
R-format: add

3. Base addressing



I-format load/S-format: lw/sw

4. PC-relative addressing



B-format/J-format/U-format: beq/jal/auipc

RV32 Extensions

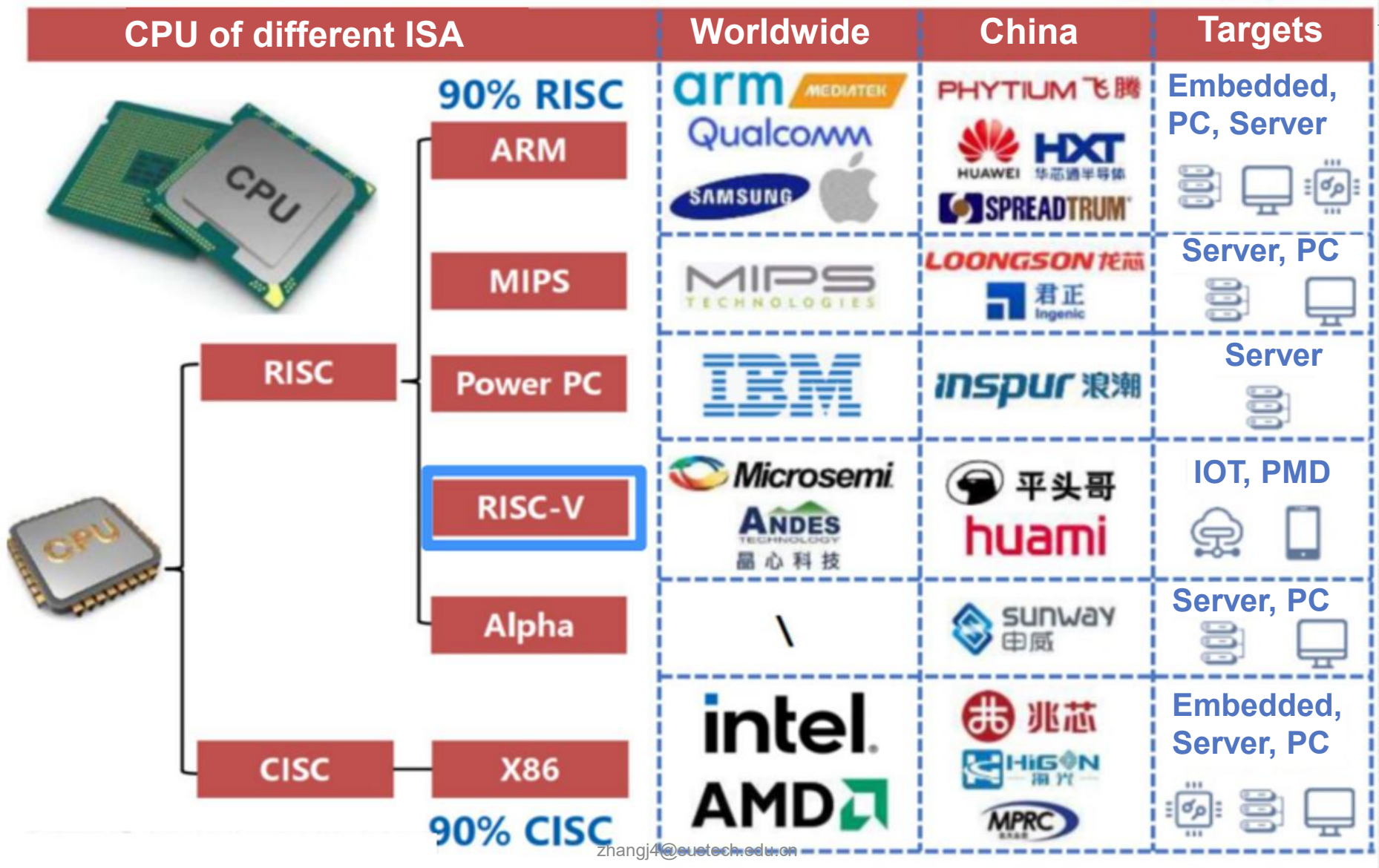
- RV32I - Base integer
- Standard extensions:
  - M – Integer Multiplication and Division
    - Extends RV32I with mul, div, rem, etc.
  - A – Atomic Instructions
    - For multi-core synchronization: amoadd.w, amoswap.w, etc.
  - F – Single-precision Floating-point (32-bit)
    - Instructions: fadd.s, fmul.s, flw, fsw
  - D – Double-precision Floating-point (64-bit)
  - C – Compressed Instructions
    - 16-bit encoded versions of common 32-bit instructions to reduce code size.
- Common combinations: RV32IM RV32IMFC

Different ISAs

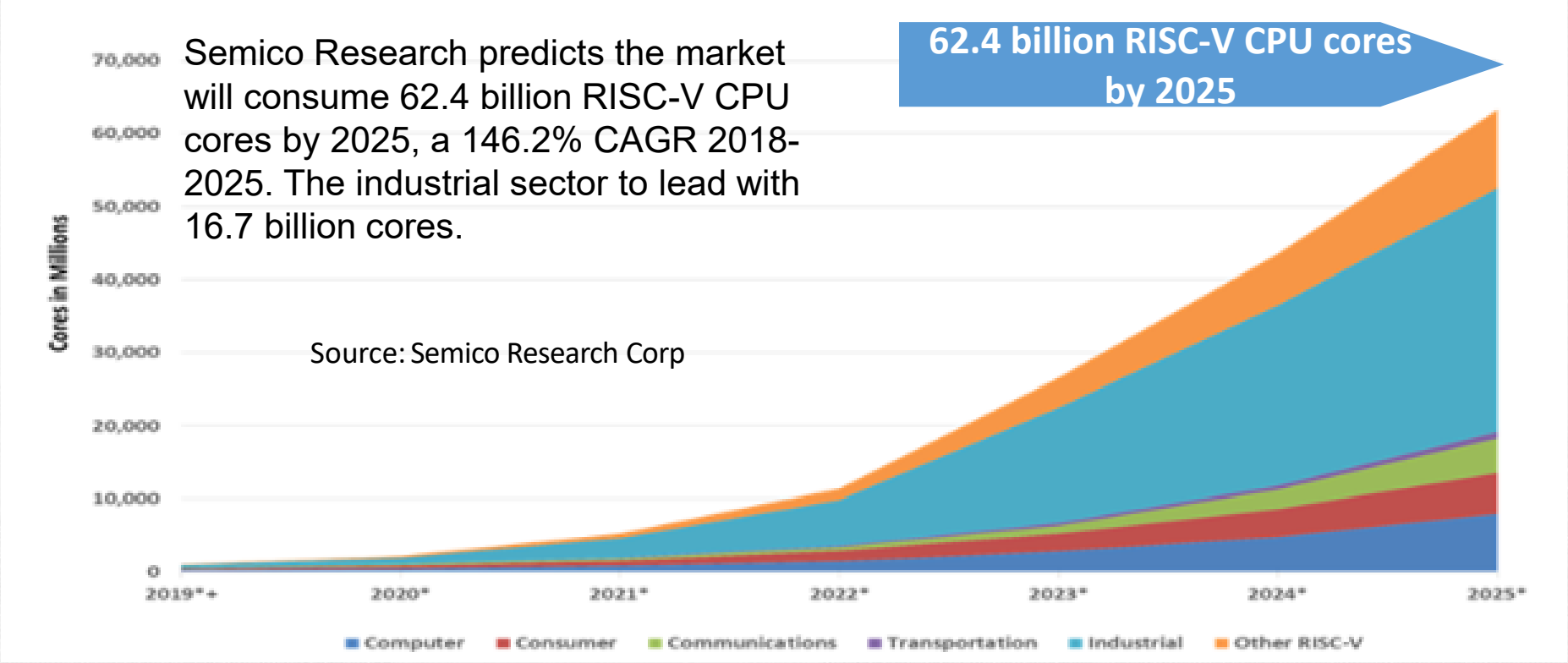
ISAs

- RISC-V
- MIPS
- ARM
- x86
- etc

RISC-V


CPU of different ISA
Worldwide
China
Targets
90% RISC
ARM
MIPS
Power PC
RISC-V
Alpha
RISC
CISC
X86
90% CISC
CPU
CPU
arm
MEDIATEK
Qualcomm
SAMSUNG
PHYTIUM 飞腾
HUAWEI
HXT
华芯通半导体
SPREADTRUM
Embedded, PC, Server
MIPS TECHNOLOGIES
LOONGSON 龙芯
君正 Ingenic
Server, PC
IBM
inspur 浪潮
Server
Microsemi
ANDES TECHNOLOGY 晶心科技
平头哥
huami
IOT, PMD
\
sunway 申威
Server, PC
intel.
AMD
兆芯
HiGON 海光
MPRC
Embedded, Server, PC

RISC-V Market Share



MIPS Instructions

- MIPS: commercial predecessor to RISC-V
- Similar basic set of instructions
  - 32-bit instructions
  - 32 general purpose registers, register 0 is always 0
  - 32 floating-point registers
  - Memory accessed only by load/store instructions
    - Consistent use of addressing modes for all data sizes
- Different conditional branches
  - For <, <=, >, >=
  - RISC-V: blt, bge, bltu, bgeu
  - MIPS: slt, sltu (set less than, result is 0 or 1)
    - Then use beq, bne to complete the branch

MIPS vs. RISC-V

- Similar basic set of instructions

	MIPS32	RISC-V (RV32)
Date announced	1985	2010
License	Proprietary	Open-Source
Instruction size	32 bits	32 bits
Endianness	Big-endian	Little-endian
Addressing modes	5	4
Registers	32 × 32-bit	32 × 32-bit
Pipeline Stages	5 stages	5 stages
ISA type	Load-store	Load-store

ARM Market share

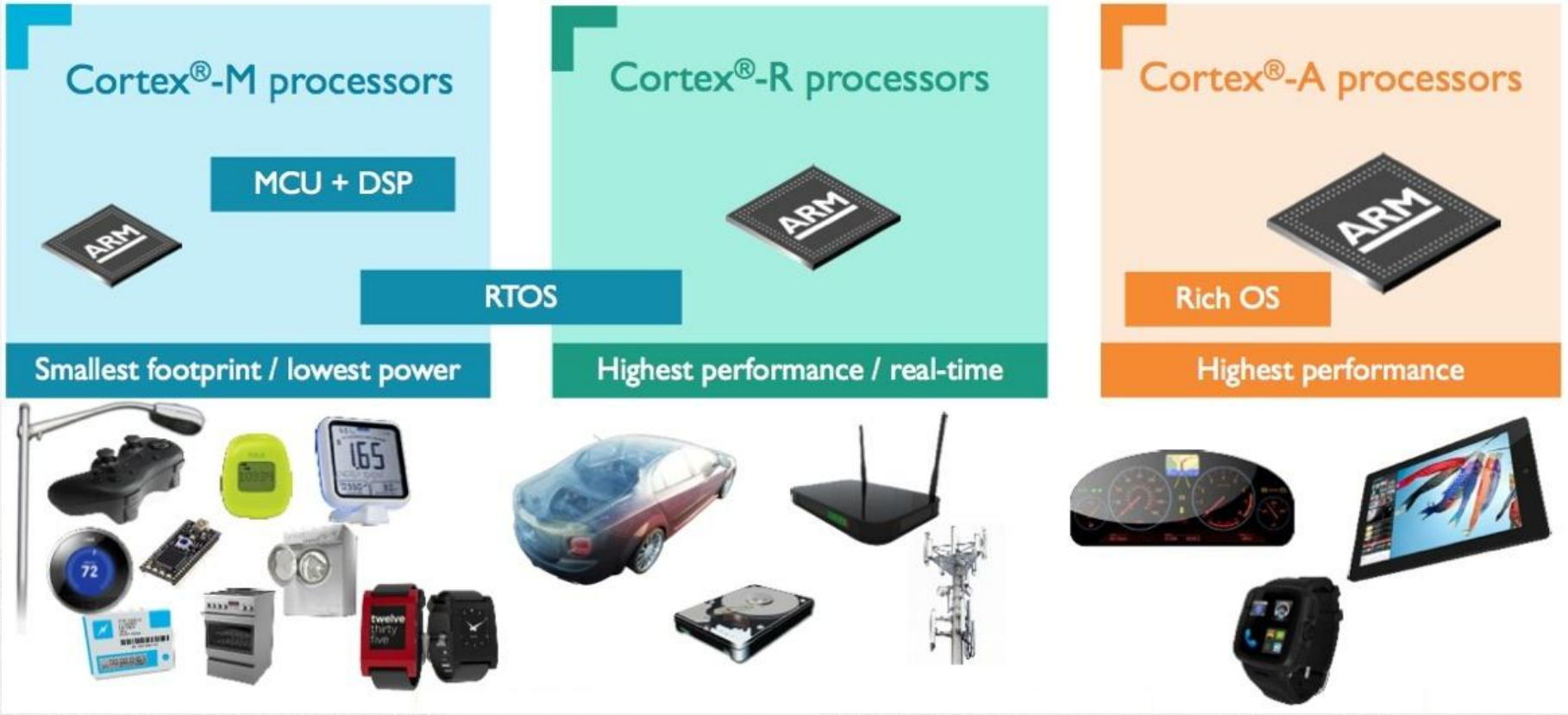
Markets for ARM in 2018

	Devices Shipped (Million of Units)	2018 Devices	Device CAGR	Chips/ Device	2018 Chips	Chip CAGR
Mobile	Smart Phone	1,900	12%	3-5	6,500	11%
	Feature & Voice Phones	400	-13%	1-2	500	-14%
	Mobile Computing* (apps only)	800	15%	1	800	15%
Home	DTV & STB	560	4%	1-4	750	5%
	Home Networking	1,500	11%	1-3	2,250	18%
	Consumer Entertainment	250	-7%	1-2	300	-10%
Enterprise	PCs & Servers (apps only)	220	2%	1	220	2%
	Enterprise Networking	1,300	3%	1-2	1,400	3%
	Computer Peripherals	800	15%	1-2	880	12%
	Hard Disk & Solid State Drives	850	6%	1	850	6%
Embedded	Automotive	3,800	5%	1	3,800	5%
	SmartCard	8,500	4%	1	8,500	4%
	Microcontroller	15,000	11%	1	15,000	11%
	Embedded Connectivity	3,000	-	1	3,000	n/a
	Others **	1,500	8%	1	1,500	8%
	Total	**40,500**	10%		**46,000**	10%

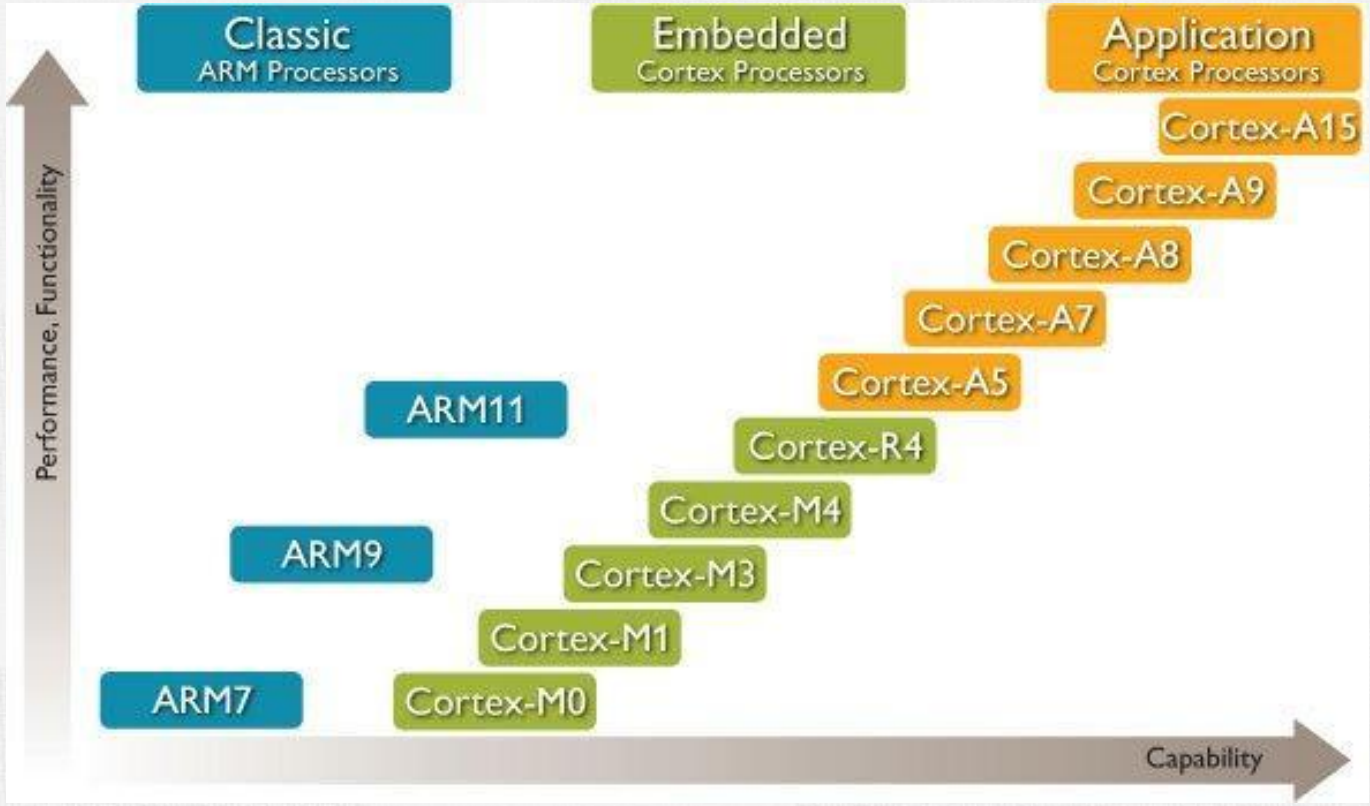
Key Growth Areas for ARM

Source: Gartner, IDC, SIA, and ARM estimates

ARM Applications



ARM CPU Series



Intel x86 Evolution: Milestones

Name	*Date*	*Transistors*	*MHz*
• 8086	1978	29K	5-10
• First 16-bit Intel processor. Basis for IBM PC & DOS			
• 1MB address space			
• 386	1985	275K	16-33
• First 32 bit Intel processor , referred to as IA32			
• Added “flat addressing”, capable of running Unix			
• Pentium 4F	2004	125M	2800-3800
• First 64-bit Intel processor, referred to as x86-64			
• Core 2	2006	291M	1060-3500
• First multi-core Intel processor			
• Core i7	2008	731M	1700-3900
• Four cores (our shark machines)			

Concluding Remarks

- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands good compromises
- Layers of software/hardware
  - Compiler, assembler, hardware
- RISC-V: typical of RISC ISAs
  - c.f. x86